Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Sunumlardan önceki son ders. Dönemin konuları Sommerville'in Software Engineering kitabının (7. baskı) özet slaytları üzerinden tekrar edildi.
- Doğrulama ve geçerleme (verification and validation, V&V): statik ve dinamik doğrulama, yazılım incelemesi (software inspection), otomatik statik analiz, temiz oda (cleanroom) yaklaşımı.
- Yazılım testi: bileşen testi (component testing), sistem testi, kusur testi (defect testing) ve geçerleme testi (validation testing), bütünlük testi, sürüm testi (release testing), kara kutu testi, ara birim testi (interface testing), test case tasarımı, test otomasyonu.
- Kalite yönetimi (quality management): süreç ve ürün kalitesi, standartlar, kalite planı, yazılım ölçümü ve metrikler.
- Konfigürasyon yönetimi: baseline, konfigürasyon yönetim planı, değişiklik yönetimi (change management), versiyon ve sürüm yönetimi.
- Süreç iyileştirme (process improvement): süreç özellikleri, süreç ölçümü, CMMI.
- Proje sunumları için son duyurular.

Hocanın Özellikle Vurguladığı Kısımlar

- Projeler bir sonraki hafta Pazartesi gününe (28 Aralık) kadar sisteme yüklenecek.
- 29 Aralık'ta sunmaya gönüllü olup Perşembe'ye kadar hocaya mail atan gruplar 10 puan fazla alacak. Gönüllü çıkmazsa hoca sunum programını hazırlayıp ders belgeleri bölümüne yükleyecek.
- Sunumda grubun tüm üyeleri bulunmalı ve yapılan bütün işleri bilmeli. Hoca rastgele soru sorabilir. Grup içinde herkes aynı notu almayacak.
- Kullanıcı testini grup kendisi, gereksinim belgesine uygun biçimde yapacak.
- Gereksinim sayısı grup büyüklüğüne göre artmalı. Örnek: 5 kişilik grupta 3 fonksiyonel gereksinim ve en az 1 fonksiyonel olmayan gereksinim test edilmeli. Fonksiyonel olmayan gereksinim sistem testiyle test edilir.
- Program kaynak kodları raporda olmayacak.
- Test aracı en az sistem testinde kullanılmalı. Birim testinde de araç kullanılacağı için hocaya göre iki yerde araç gerekir.
- Sunum için video kaydı hazırlanabilir. Geçen yıl yazılım mühendisliği dersinde bunu yapanlar olmuştu.
- Inspection statik doğrulamadır (static verification). Test dinamik doğrulamadır (dynamic verification).
- Inspection ile test birbirinin alternatifi değildir. İkisi birlikte kullanılmalıdır.
- Fonksiyonel olmayan özellikler (performans, kullanılabilirlik) inspection ile değil, ancak çalıştırarak, yani testle gösterilebilir.
- "Ölçemediğinizi kontrol edemezsiniz."

Kısa Tekrar Notları

- Doğrulama (verification): "Ürünü doğru yolla mı oluşturuyoruz?" Spesifikasyonlara uygunluk.
- Geçerleme (validation): "Doğru ürünü mü oluşturuyoruz?" İstenen özellikler sağlanıyor mu?

- V&V yaşam döngüsünün her adımında uygulanmalıdır. Amaç yazılımı tamamen hatasız yapmak değil, amacına yetecek kadar hatasız yapmaktır.
- Debug: test sonuçlarından hatanın yerini bul, onarımı tasarla, hatayı düzelt, programı yeniden test et.
- Inspection için ön koşullar: ayrıntılı spesifikasyon, kurum standartlarını bilen ekip, hata kontrol listeleri (checklist).
- Inspection rolleri: yazar, inceleyici, okuyucu, moderatör. Bir kişi birden fazla rol üstlenebilir.
- Bileşen testinden geliştirici, sistem testinden bağımsız test ekibi sorumludur.
- Kusur testinde başarı, bulunan hata sayısıyla ölçülür. Geçerleme testi gereksinimlerin karşılandığını gösterir.
- Test süreci: test case tasarımı, test verisi hazırlama, programı test verisiyle çalıştırma, sonuçları beklenenle karşılaştırma, rapor.
- Yukarıdan aşağıya bütünlük testi mimari hataları bulmada daha iyidir.
- Test case tasarım yaklaşımları: gereksinim tabanlı test, bölümleme testi (partition testing), yapısal test (structural testing).
- Temel yol sayısı = akış grafındaki kapalı bölge sayısı + 1.
- Süreç kalitesi: geliştirme süreci kurallara uygun mu? Ürün kalitesi: ürün gereksinimleri sağlıyor mu?
- Kalite planı kurum geneli içindir. Test planı her proje ve alt ürünleri içindir.
- Baseline: sürecin belli bir aşamasında hangi yazılım elemanlarının bulunduğunu gösterir.
- Süreç iyileştirme döngüsü: süreci ölç, analiz et, gerekirse değiştir.
- CMMI düzeyleri: 0 eksik, 1 gerçekleştirilmiş (performed), 2 yönetilen (managed), 3 tanımlı (defined), 4 nicel yönetilen (quantitatively managed), 5 iyileştiren (optimizing).

Detaylı Açıklamalar

Doğrulama ve Geçerleme

- Hedef: sistemin fonksiyonları, kullanıcı beklentileri ve pazarlanacak ortamla uyumlu bir yazılım geliştirmek.
- Statik doğrulama: yazılım incelemesi. Program çalıştırılmadan, gerekirse araç desteğiyle yapılır.
- Dinamik doğrulama: test. Ürün çalıştırılarak davranışı gözlenir.
- Inspection ile incelenebilecek ürünler: gereksinim spesifikasyonu, yüksek seviye tasarım, formal spesifikasyon, ayrıntılı tasarım, program.
- Program testi ise program ve prototip üzerinde yapılır.
- Kusur testi (defect testing): sistemde hata var mı diye bakar. Birim, bütünlük ve bileşen testlerinde yapılır. Bu testler daha çok doğrulamaya hizmet eder.
- Geçerleme testi (validation testing): gereksinimler karşılanıyor mu diye bakar. Fonksiyonel olmayan gereksinimler daha çok sistem testinde ve kullanıcı testinde ele alınır.

Debug Süreci

```
Test sonuçları
  -> Hatanın yerini bul
  -> Hata onarımını tasarla (spesifikasyona bakarak)
  -> Hatayı onar
  -> Programı yeniden test et (test case'lerle)
```

- Test ile debug ayrı süreçlerdir. Test hatanın varlığını gösterir. Debug hatanın yerini bulur ve düzeltir.

V&V Planlaması ve V Modeli

- Yazılım geliştirme için plan yapıldığı gibi doğrulama ve geçerleme için de plan yapılır.
- Plan, statik doğrulama ile test arasında denge kurar ve test sürecinin standartlarını belirler.
- V modeli bu kez yatay çizilmiş olarak gösterildi.

Geliştirme aşaması	Bu aşamada hazırlanan plan	İlgili test
Gereksinim spesifikasyonu	Kabul testi planı	Kabul testi
Sistem spesifikasyonu	Sistem bütünlük testi planı	Sistem bütünlük testi
Sistem tasarımı	Alt sistem bütünlük testi planı	Alt sistem bütünlük testi
Ayrıntılı tasarım ve kod	-	Modül ve birim testi

- Test sırası: birim testi, alt sistem bütünlük testi, sistem bütünlük testi, kabul testi.

Test Planının Yapısı

- Test süreci.
- Gereksinim izlenebilirliği (requirements traceability).
- Test edilecek parçalar.
- Test takvimi (schedule), örneğin Gantt diyagramı ile.
- Test kayıt süreçleri: testleri çalıştırıp sonuç göstermek yetmez. Sonuçlar sistematik olarak kaydedilmeli ve raporlanmalı.
- Donanım ve yazılım gereksinimleri: test için araç veya donanım kullanıldı mı?
- Test sürecindeki kısıtlar.

Yazılım İncelemesi (Inspection)

- Tek bir incelemede birçok hata birlikte bulunabilir. Testte ise bir hata başka hataları gizleyebilir.
- Inspection'da spesifikasyonla uygunluğa bakılır. Müşterinin gerçek gereksinimleriyle uyuma ise testle bakılır.
- Inspection ile kod hataları, tasarım hataları, anomaliler, mantık hataları ve standartlara uymayan yerler bulunur.
- Süreç: planlama, hızlı gözden geçirme (overview), bireysel hazırlık, inceleme toplantısı, yeniden çalışma (rework).
- Herkes kendisine verilen modülü inceler. Sonra toplantıda tüm modüllerin üzerinden birlikte geçilir.
- Fagan incelemesinde ekiplerin kuruluşu ve adımlar daha ayrıntılıdır.

Roller

Rol	Görev
Yazar	Programı veya tasarımı yazan kişi
İnceleyici	Hataları bulmakla görevli
Okuyucu	Toplantıda incelenen kodu okuyan kişi
Moderatör	Süreci yöneten kişi

Kontrol Listesi Örnekleri

- Veri hataları: isimlendirme, karakter dizisi kullanımı, taşma (overflow).
- Kontrol hataları: koşul ifadesi doğru mu, sonsuz döngü var mı?
- Girdi-çıktı hataları.
- Ara birim hataları.
- Bellek yönetimi hataları.
- Bir bölümde sık çıkan hataların listelenmesi sonradan istatistik yapmayı da kolaylaştırır.

İnceleme Hızları

Aşama	Hız (saatte ifade)
Hızlı gözden geçirme	Yaklaşık 500
Bireysel hazırlık	Yaklaşık 125

Aşama	Hız (saatte ifade)
İnceleme toplantısı	Yaklaşık 125

- Bu nedenle inspection pahalıdır. Hoca bunun yaklaşık 40 adam-saate denk geldiğini söyledi.

Otomatik Statik Analiz

- Statik analiz araçları programı inceler ve olası hata koşullarına V&V ekibinin dikkatini çeker. Maliyeti düşürmek için genellikle kullanılır.
- Aşamalar: kontrol akışı analizi, veri kullanım analizi, ara birim analizi, yol analizi (tüm program yollarının ve kontrol deyimlerinin araştırılması).
- Slayttaki araç eskidir. Günümüzde çok sayıda farklı araç vardır.
- Formal metotlar da kullanılabilir. Hoca ayrıntıya girmedi.

Temiz Oda (Cleanroom) Yazılım Geliştirme

- Hata kritik yazılımlarda kullanılır.
- Formal spesifikasyon yapılır. Statik doğrulama ve inspection ağırlıklı kullanılır.
- Amaç güvenilirliği (reliability) artırmak ve hatayı en aza indirmektir.

Yazılım Testi

- Bileşen testi: birim testi ve onun bir üstündeki bileşenlerin, yani tek bir program parçasının test edilmesi. Geliştiricinin sorumluluğundadır ve geliştiricinin deneyimine dayanır.
- Sistem testi: birden fazla bileşenin veya alt sistemin birleşimine uygulanır. Bağımsız bir test ekibinin sorumlu olması daha uygundur. Sistem spesifikasyonları test edilir.
- Bileşen testinden çıkan birimler önce bütünlük testiyle birleştirilir, sonra sistem testi yapılır.

Kusur Testi ve Geçerleme Testi

	Geçerleme testi	Kusur testi
Amaç	Geliştiriciye ve müşteriye gereksinimlerin karşılandığını göstermek	Spesifikasyona uymayan hatalı davranışları bulmak
Başarı ölçütü	Sistem beklenen kullanımda doğru çalışır	Ne kadar çok hata bulunursa test o kadar başarılı

Test Süreci

```
Test case tasarımı -> Test verisi hazırlama -> Programı test verisiyle çalıştırma
  -> Test sonuçlarını beklenen sonuçlarla karşılaştırma -> Test raporu
```

- Sistem testinde test politikaları:
  - Menülerden erişilen tüm fonksiyonlar denenir.
  - Birlikte çalışan fonksiyon kombinasyonları denenir.
  - Kullanıcı girdisi verildiğinde sonucun doğru olup olmadığına bakılır.

Bütünlük Testi

- Yukarıdan aşağıya (top-down) veya aşağıdan yukarıya (bottom-up) yapılabilir. Her iki durumda da sistem artımlı (incremental) olarak birleştirilir.
- Artımlı bütünlük testinde testler (T1, T2, T3…) her yeni bileşen eklendiğinde yeniden çalıştırılır.
- Yukarıdan aşağıya test sistem mimarisindeki hataları bulmakta daha iyidir.
- Aşağıdan yukarıya testte önceden test edilmiş modüller birleştirilerek yukarıya çıkılır.
- Bütünlük testinde test ekibi kaynak koda erişebilir.

Sürüm Testi (Release Testing)

- Müşteriye sunulacak sürümün bütün olarak test edilmesidir.
- Dağıtımdan önce yapılır. Sistemin gereksinimleri karşıladığını gösterir.
- Genellikle kara kutu (black box) yani fonksiyonel testtir. Müşteri spesifikasyonlarına dayanır.

Kara Kutu Testi Kuralları

- Sistemin iç yapısı bilinmeyebilir. Ne yapması gerektiği bilinir. Girdiler verilip uygun çıktıların alınıp alınmadığına bakılır.
- Hata mesajlarını tetikleyecek girdiler seçilmelidir.
- Aynı girdi veya girdi dizisi birkaç kez tekrarlanmalıdır.
- Geçersiz veri girilip çıktıları kontrol edilmelidir.

Senaryo ve Use Case Tabanlı Test

- Önce kullanıcının ağzından sistemin nasıl işleyeceğini anlatan bir test senaryosu yazılır. Slayttaki örnek bir kütüphane sistemidir.
- Senaryodan test edilmesi gerekenler çıkarılır:
  - Doğru ve hatalı kullanıcı bilgisini ayırt eden giriş (login) mekanizması.
  - Sistemin belgeler hakkında doğru bilgi verip vermediği.
  - Bilgiyi indirmek için gerekli izin mekanizmasının olup olmadığı.
- Use case'lerden ve ardışıl (sequence) diyagramlardan hangi testlerin yapılacağı çıkarılabilmelidir.

Performans ve Stres Testi

- Performans testi sürümden önce yapılan sistem testlerinden biridir. Cevap süresi ve güvenilirlik test edilir.
- Stres testi (dayanıklılık testi) sistemin taşıyabileceği en yüksek yükü bulur.
- Dağıtık sistemlerde ağın ve kullanıcı sayısının dayanıklılığı test edilir.

Bileşen Testi

- Bir kusur testidir.
- Her bileşende işlemler, fonksiyonlar, sınıf tanımları, özellikler, metotlar ve kullanıcı ara birimleri doğru çalışıyor mu test edilir.
- Nesnelerin ve sınıfların doğru oluşturulduğu da test edilmelidir.
- Slaytta örnek olarak bir hava durumu istasyonu nesnesinin testi verildi.

Ara Birim Testi

- Ara birim türleri:
  - Parametre geçişi.
  - Paylaşımlı bellek.
  - Prosedürel ara birimler (alt sistemler arası çağrılar).
  - Mesaj geçişi.
- Ara birim hataları:
  - Ara birimin yanlış kullanımı veya eksik olması.
  - Parametrelerin yanlış sırada verilmesi.
  - Ara birimin mantıksal olarak yanlış anlaşılması.
  - Zamanlama hataları.
- Test yöntemleri:
  - Parametreleri uç değerlerde ve aralık sınırlarında seçmek.
  - Bileşenin hatalı çalışmasına yol açacak testler tasarlamak.
  - Mesaj geçişli sistemlerde stres testi yapmak.

Test Case Tasarımı

- Amaç etkin geçerleme ve kusur testi yapacak test case'ler üretmektir.
- Yaklaşımlar: gereksinim tabanlı test, bölümleme testi, yapısal test.

Gereksinim Tabanlı Test

- Spesifikasyondaki her gereksinimin sağlanıp sağlanmadığı test edilir. Öğrenciler projelerinde bunu yapacak.
- Örnek: veri tabanında kullanıcı arama. Mevcut bir kullanıcı ve mevcut olmayan bir kullanıcı arandığında sistemin hangi yanıtı verdiğine bakılır.

Bölümleme Testi (Equivalence Partitioning)

- Bir kara kutu test tekniğidir.
- Girdi ve çıktılar sınıflara bölünür. Geçerli ve geçersiz girdiler ayrı sınıflardır.
- İç yapı bilinmediği için test verisi dışarıdan, bu sınıflar üzerinden seçilir.
- Test sayısı azsa üç sınıf yeterlidir. Daha çok test yapılabiliyorsa aralıklar küçültülür.
- Slayttaki örnek (her sınıftan ve sınırlardan değer seçilir):

Girdi	Sınıflar	Seçilen test değerleri
Girdi değeri sayısı	4'ten az, 4-10 arası, 10'dan çok	3, 4, 7, 10, 11
Girdi değerleri	10000'den küçük, 10000-99999 arası, 99999'dan büyük	9999, 10000, 50000, 99999, 100000

- Aynı yöntem bir arama rutininin (ikili arama) test verisini oluşturmak için de kullanılır.

Yapısal Test (Structural Testing)

- Yapısal test, saydam kutu testi ve white box test aynı anlamda kullanılır.
- Test case'ler program yapısına göre seçilir. Her ek test case programın yeni bir dalını izler.
- Amaç tüm program ifadelerini, yani tüm temel yolları test etmektir.

Yol Testi: İkili Arama Örneği

- Algoritmanın akış grafı (flow graph) çizilir.
- Temel (bağımsız) yol sayısı = kapalı bölge sayısı + 1. İkili arama örneğinde 3 kapalı bölge vardır, temel yol sayısı 4'tür.
- Slayttaki akış grafının yapısı:

```
1 -> 2 -> 3 -> 4 -> 5
5: bottom > top          -> 14 (çıkış)
5: while bottom <= top   -> 6 -> 7
7: elemArray[mid] = key  -> 8 -> 9 -> 10 -> 14
7: elemArray[mid] != key -> 11
11: elemArray[mid] > key -> 12 -> 5
11: elemArray[mid] < key -> 13 -> 5
```

- Bağımsız yollar:
  - 1, 2, 3, 4, 5, 14 (döngüye hiç girilmez).
  - 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 14 (anahtar bulunur).
  - 1, 2, 3, 4, 5, 6, 7, 11, 12, 5, … (alt yarıya geçilir).
  - 1, 2, 3, 4, 5, 6, 7, 11, 13, 5, … (üst yarıya geçilir).
- Her yol en az bir kez çalıştırılmalıdır.
- Yolların çalışıp çalışmadığını görmek için dinamik program analizcisi kullanılabilir.

Test Otomasyonu

- Büyük yazılımlarda test verisini hazırlama ve çalıştırma sürecini kısaltmak ve maliyeti düşürmek için test otomasyonu kullanılır.
- Örnek: Java için JUnit.
- Test araçları test betikleri (script) oluşturur, kaynak kod üzerinde dinamik analiz yapar ve test raporu üretir.

Kalite Yönetimi

- Kalite yönetimi kurumda bir kalite yönetim ekibiyle sağlanır.
- Kalite özellikleri McCall modelinde ayrıntılı incelenmişti. ISO 9126'da da kalite özellikleri tanımlıdır.
- Kurumda bir kalite kültürü olmalıdır: ekipte kim hangi kaliteden sorumlu?
- Kalite: yazılımın kullanımında müşteri ve geliştirici açısından sağlaması gereken özellikler ve bunların sağlanma derecesidir. Spesifikasyonlara uygunluk derecesi de kalitenin parçasıdır.
- Büyük ve karmaşık sistemlerde kalite yönetimi daha önemlidir ve ayrıntılı kalite belgeleri gerekir. Küçük sistemlerde daha az belge ve daha az kişiyle yürütülür.

Kalite Yönetimi Aktiviteleri

Aktivite	Açıklama
Kalite güvencesi (quality assurance)	Kurumda uygulanacak standartların ve süreçlerin belirlenmesi
Kalite planlama	Her proje için bu standartlara göre ayrı plan
Kalite kontrolü	Geliştirme ekibinin standartlara uyup uymadığının kontrolü

- Kalite yönetim ekibi geliştirme sürecini takip etmekle görevlidir.

Süreç Kalitesi ve Ürün Kalitesi

- Süreç kalitesi: geliştirme ekibi kurallara uyuyor mu, hazırladığı belgeler standartlara uygun mu?
- Ürün kalitesi: ürün istenen gereksinimleri ve kalite özelliklerini sağlıyor mu?
- Süreç tabanlı kalite: şelale (waterfall) modelindeki her adımda (gereksinim, tasarım, gerçekleştirme, test) yazılım mühendisliği kurallarına uyuluyor mu?

```
Süreci tanımla -> Ürünü geliştir -> Ürün kalitesini değerlendir
  kalite yeterli  -> süreç uygun yürütülmüştür
  kalite yetersiz -> süreci iyileştir ve tekrarla
```

Standartlar

- Ürün standartları: yazılım ürününün sağlaması gereken özellikler ve belge formatları.
- Süreç standartları: geliştirme sürecinin nasıl yürütüleceği. Örnek: gereksinim spesifikasyon standardı, tasarım standardı, konfigürasyon yönetim planı standardı, test planı standardı, değişiklik kontrol süreci.
- Hem kurum standartları hem proje standartları olmalıdır.
- Standartların sorunu: süreci daha bürokratik yapabilirler.
- ISO 9000: kalite yönetimi için genel bir çerçeve. ISO 9001: kurumlarda geliştirme ve bakım süreçlerine uygulanan kalite sistemi tanımı.
- IEEE standartları: belgelerin nasıl hazırlanacağını belirten belgeleme standartları vb.

Kalite Planı ve Test Planı

- Kalite planı: kurumun kalite özellikleri, değerlendirme süreci, kurumsal standartlar, ürün planları, süreç tanımları, kalite hedefleri, risk yönetimi.
- Kalite planı kurum geneli için daha genel bir süreç anlatır.

- Test planı her proje ve projenin alt ürünleri için hangi özelliklerin sağlanacağını gösterir.

Kalite Özellikleri

- Slayttaki liste (ISO 9126 ve McCall ile örtüşür): güvenlik, güvenilirlik (reliability), sağlamlık (robustness), anlaşılabilirlik, test edilebilirlik, uyarlanabilirlik, modülerlik, karmaşıklık, yeniden kullanılabilirlik, kullanışlılık, taşınabilirlik, verimlilik.

Yazılım Ölçümü ve Metrikler

- Kaliteyi kontrol etmek için ölçmek gerekir.
- Kalite gözden geçirmeleri (quality review): programların, belgelerin ve ilerlemenin standartlara uygunluğunun gözden geçirilmesi.
- Yazılım metriği: yazılımın bir özelliğini ölçmek için kullanılan birimdir. Örnek: kod satırı sayısı (lines of code), kişi-gün sayısı (person-days).
- Hem ürün hem süreç üzerinde ölçüm yapılır. Ölçüm sonuçlarına göre yönetim kararlar verir ve süreç değişir.

İç ve Dış Özellikler

- Dış kalite özellikleri doğrudan ölçülemez. Ölçülebilen iç özelliklerle ilişkilendirilir.

Dış özellik	İlgili iç özellikler
Bakım yapılabilirlik (maintainability)	Prosedürlerdeki parametre sayısı, çevrimsel karmaşıklık (cyclomatic complexity), kod satırı sayısı, kullanıcı el kitabının uzunluğu
Güvenilirlik (reliability)	Çevrimsel karmaşıklık, program uzunluğu, hata mesajı sayısı
Taşınabilirlik (portability)	Program uzunluğu
Kullanılabilirlik (usability)	El kitabının kapsamı, hata mesajı sayısı

Ölçüm Süreci

```
Yapılacak ölçümleri seç -> Değerlendirilecek bileşenleri seç
  -> Bileşen özelliklerini ölç -> Aykırı ölçümleri belirle
  -> Aykırı bileşenleri analiz et
```

- Ölçüm için ürün ve süreç verisi toplanır. Veri toplama mümkünse otomatik yapılır.
- Ürün metrikleri:
  - Dinamik metrikler: program çalıştırılarak elde edilir. Etkinlik ve güvenilirliği değerlendirir.
  - Statik metrikler: sistemin gösterimlerinden (kod, tasarım) elde edilir. Karmaşıklık, anlaşılabilirlik ve bakım yapılabilirliği değerlendirir.
- Bağlaşım (coupling) yapısal programlamada birimler arasındaki ilişkiyle ve modülerlikle ilişkilidir. Nesneye yönelik yaklaşımda sınıflar arasındaki ilişkiyle ilgilidir. Bu yüzden iki yaklaşımın metrikleri aynı isimlerle ifade edilebilir.
- Ölçüm analizinde beklenmedik sonuçlar (sürprizler) çıkabilir.

Konfigürasyon Yönetimi

- Kalite yönetiminin bir parçasıdır.
- Yazılım sisteminin yeni versiyonları oluştukça bunların takibi, özel kullanıcı gereksinimlerinin takibi ve sistem değişikliklerinin kaydı gerekir.
- Konular: konfigürasyon yönetim planlaması, değişiklik yönetimi, versiyon ve sürüm yönetimi, CASE araçları.

Baseline

- Sürecin belli bir aşamasında hangi temel ürünlerin bulunduğunu gösterir.

Aşama sonu	Baseline'daki ürünler
Gereksinim analizi	SRS belgesi, proje planı
Tasarım	Tasarım diyagramları, veri tabanı diyagramları (ER)
Gerçekleştirme	Her modülün kodu, birim testi test case'leri

- Sistem üzerinde yapılan her değişiklik de yeni bir baseline kabul edilir.
- Sistem aileleri: başlangıç sistemi ve farklı bilgisayarlarda veya işletim sistemlerinde çalışan versiyonları.

Konfigürasyon Yönetim Planı

- Büyük bir sistemde binlerce belge üretilir: planlar, spesifikasyonlar, tasarımlar, programlar, test verisi, kullanıcı el kitapları.
- Plan şunları belirler:
  - Belgelerin isimlendirme şeması (hiyerarşik isimlendirme).
  - Konfigürasyon yönetim süreçlerinin sorumluları.
  - Değişiklik kontrolü ve versiyon yönetimi kuralları.
  - Bakımın nasıl yapılacağı.
- Bir kod, bir test verisi, bir veri tabanı tablosu birer konfigürasyon maddesidir (configuration item).
- Konfigürasyon yönetimi için IEEE standardı vardır.
- Konfigürasyon yönetim veri tabanı: her eleman tipi hakkında bilgi tutar. Büyük projelerde elemanlar oluşturulurken bir CASE aracıyla veri tabanına girilir. Sonradan girmek pratikte mümkün değildir.

Değişiklik Yönetimi

- Değişiklik istekleri geliştiricilerden, kullanıcılardan veya pazar baskısından gelebilir.
- Değişikliği kimin yaptığı ve değişikliğin ne olduğu konfigürasyon yönetim aracına kaydedilir.

```
Değişiklik istek formunu doldur
  -> İsteği analiz et
  -> İstek geçerliyse: nasıl değiştirileceğini bul, maliyetini hesapla
  -> Maliyete göre değişikliğin yapılıp yapılmayacağına karar ver
  -> Kabul edilirse: değişikliği yap, yeni sistem versiyonu oluştur
```

- Değişiklik istek formu geliştirme başladığı andan itibaren kullanılabilir. Teslimden sonraki değişiklikler daha önemli ve daha maliyetlidir.
- Değişikliğin ne zaman yapıldığı ve isteğin kapanıp kapanmadığı izlenir.

Versiyon ve Sürüm Yönetimi

- Versiyon: işlevsel olarak diğerlerinden farklılık gösteren bir sistem örneği.
- Sürüm (release): kullanıcılara dağıtılan versiyon.
- Versiyon numaralandırma: küçük değişikliklerde 1.1, 1.2; daha önemli değişiklikte 2.0, 2.1 gibi. Numaralar bir türetme ağacı oluşturur.
- Özellik tabanlı (attribute-based) ve değişiklik tabanlı (change-oriented) tanımlama da kullanılabilir.
- Sürüm problemleri:
  - Eski sürümden yeni sürüme geçişte aksaklık olmaması.
  - Kullanım süreci çok değiştiyse kabul ve alışma süreci.
  - Verinin kayıpsız aktarılması.
- Sürüm stratejisini etkileyenler: sistemin teknik kalitesi, platform değişiklikleri, pazarlama gereksinimleri, müşterinin değişiklik önerileri.
- Sistem oluşturma (system building): bileşenlerin paketlenmesi ve bütünleştirilmesi.

- Konfigürasyon yönetimi için CASE aracı kullanılmalıdır. Proje yönetim araçlarının çoğunda bulunur. Yalnızca değişiklik yönetimi için kullanılan versiyon yönetim sistemleri de vardır.

Süreç İyileştirme

- Amaç yazılım geliştiren kurumların daha iyi hizmet vermesidir.
- Süreç özellikleri:
  - Anlaşılabilirlik: süreç tanımı ve belgeleri anlaşılır mı?
  - Görünürlük: süreç işi yapanlarca izlenebilir mi?
  - Desteklenebilirlik: CASE araçları kullanılmış mı?
  - Kabul edilebilirlik.
  - Güvenilirlik: hata oranı ne kadar?
  - Sağlamlık: beklenmeyen problemlere rağmen süreç sürdürülebiliyor mu? Risk analizi yapılmış mı?
  - Bakım yapılabilirlik ve hız.

```
Süreci ölç -> Süreci analiz et -> Gerekirse süreci değiştir -> (tekrar ölç)
```

- Ürün kalitesini etkileyen faktörler: geliştirme teknolojisi, geliştiren insanların kalitesi, süreç kalitesi, zaman ve maliyet.
- Süreç kalitesi örneği: gerekli işlemler yapılmış mı, yoksa tasarımı atlayıp kodlamaya mı geçilmiş?
- Küçük sistemlerde ve prototiplerde informal süreçler yeterli olabilir. Büyük sistemlerde yönetilebilir ve iyi anlaşılan süreçler gerekir.
- Süreç araç desteği: konfigürasyon yönetimi aracı, proje yönetimi aracı, analiz ve tasarım platformları kullanılmış mı?
- Süreç ölçümü: hedefler belirlenir, hedefleri değerlendirecek sorular ve ölçüm birimleri seçilir.
- Süreç modeli incelenirken bakılan elemanlar: aktiviteler, süreçler, ara ve son ürünler (deliverables), koşullar, roller, sorumluluklar, iletişim.
- Örnek: modül testi aktivitesinde ön koşullar sağlandı mı, test mühendisleri test modüllerini oluşturdu mu?

CMMI

- CMM 1990'larda, CMMI 2000'lerde Software Engineering Institute (SEI) tarafından ortaya konmuştur.
- İlk CMM düzeyleri: başlangıç (initial), tekrarlanabilir (repeatable), tanımlı (defined), yönetilen (managed), iyileştiren (optimizing).
- CMM'de farklı modeller ayrı ayrı yürütülüyordu. CMMI bunları bütünleştirdi. Süreç alanları, hedefler ve pratikler tanımlandı.
- CMMI'de süreç yönetimi, proje yönetimi, mühendislik ve destek alanları birleştirildi.
- CMMI hedefleri: projeyi doğru izlemek ve kontrol etmek, tüm gereksinimleri analiz etmek, gerekli işlevlerin geliştirilmesini sağlamak.

Düzey	Ad
0	Eksik (incomplete)
1	Gerçekleştirilmiş (performed)
2	Yönetilen (managed)
3	Tanımlı (defined)
4	Nicel yönetilen (quantitatively managed)
5	İyileştiren (optimizing)

Ders sonrası notlar

- Hoca "kaç kapalı bölge varsa bir ekleyerek temel yol sayısı bulunur" dedi. Bu, çevrimsel karmaşıklığın $V(G) = E - N + 2$ formülüyle aynı sonucu verir. İkili arama grafında $E = 16$ kenar ve $N = 14$ düğüm var: $V(G) = 16 - 14 + 2 = 4$.

- Hoca “ISO 9000 alt çerçevesinde 9100-9126 gibi kalite tanımları” dedi. ISO 9126 ISO 9000 ailesine ait değildir. ISO/IEC 9126 ayrı bir yazılım ürün kalitesi standardıdır ve yerini ISO/IEC 25010 almıştır.
- Hoca “CMMI’de gereksinim yönetimi, proje planlama, izleme, ölçüm ve analiz birinci aşamada vardı” dedi. Aşamalı (staged) CMMI modelinde bu süreç alanları 2. düzeye (managed) aittir. 1. düzeyde (initial) tanımlı süreç alanı yoktur.
- Hoca “CMMI düzeyleri 0’dan 5’e” sıraladı. 0-5 sıralaması CMMI’nin sürekli (continuous) gösterimine aittir. Aşamalı (staged) gösterimde düzeyler 1-5’tir: initial, managed, defined, quantitatively managed, optimizing.
- İnceleme hızlarındaki “40 adam-saat” Sommerville’de yaklaşık 500 satırlık kodun incelenmesi için verilen iş gücüdür.
- Altyazıda “Summerville” diye geçen Sommerville, “MacCol” ve “Makkol” diye geçen McCall, “Ecolons Partitioning” diye geçen equivalence partitioning, “Yuz kez” diye geçen use case, “data seconds diagramı” diye geçen sequence diyagramıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.